

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 18: COMPARANDO SOMA DE VETORES ENTRE CPU E GPU EM OPENMP

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 24/09/2026.

1. EXPERIMENTO

Faça os exercícios de adição de vetores, vadd.c, dos slides 27 e 48 do tutorial de programação de GPUs com OpenMP em um dos nós com GPU do NPAD. Compare os tempos de execução somente na CPU e com o uso da GPU. Reporte seu progresso apresentando os problemas encontrados e as soluções apresentadas.

2. CONCEITOS RELEVANTES

- **CPU × GPU**: a CPU tem menos núcleos, porém mais versáteis e com acesso direto à RAM. A GPU tem muitos núcleos simples, ideal para aplicar a mesma operação em milhões de elementos.
- **Memórias separadas:** normalmente CPU e GPU não compartilham a mesma memória física. Os vetores nascem na RAM; para a GPU processá-los, precisam ser copiados para a VRAM. Esse é o custo que pesou no resultado.
- **Transferência de dados:** mover dados por PCIe pode ser relativamente lento. Para uma soma simples, transferir centenas de MB pode custar mais que executar a soma na CPU.

3. MÁQUINA UTILIZADA

No NPAD, para realizar os testes relacionados à tarefa, foi utilizada a partição *gpu-8-v100*.

4. CÓDIGO

Figura 1: Código paralelizado (sem GPU)

```
#include <stdio.h>
#include <omp.h>
#define tamanhoVetor 100000000
#define tolerancia 0.0000001f

int main()
{

   float vetorA[tamanhoVetor], vetorB[tamanhoVetor];
   float vetorSoma[tamanhoVetor], vetorEsperado[tamanhoVetor];
   int quantidadeErros = 0;

   double tempoInicializacao, tempoCalculo, tempoTeste;
   tempoInicializacao = -omp_get_wtime();

  // Inicializa os vetores de entrada, soma e resultado esperado.
  #pragma omp parallel for
  for (int indice = 0; indice < tamanhoVetor; indice++){
     vetorA[indice] = (float)indice;
     vetorB[indice] = 2.0f * (float)indice;
     vetorSoma[indice] = 0.0f;
     vetorEsperado[indice] = indice + 2 * indice;
  }

  tempoInicializacao += omp_get_wtime();
  tempoCalculo = -omp_get_wtime();

  // Soma os vetores de entrada e armazena o resultado.
  #pragma omp parallel for
  for (int indice = 0; indice < tamanhoVetor; indice++){
     vetorSoma[indice] = vetorA[indice] + vetorB[indice];
  }
```

```
  tempoCalculo += omp_get_wtime();
  tempoTeste = -omp_get_wtime();

  // testando se o resultado da soma está correto
  #pragma omp parallel for reduction(+:quantidadeErros)
  for (int indice = 0; indice < tamanhoVetor; indice++) {
    float diferenca = vetorSoma[indice] - vetorEsperado[indice];
    diferenca = diferenca * diferenca;
    if (diferenca > tolerancia) quantidadeErros++;
  }

  tempoTeste += omp_get_wtime();

  printf("Vetores somados com %d erros\n", quantidadeErros);
  printf("Tempo de inicialização: %.3fs\n", tempoInicializacao);
  printf("Tempo de computação: %.3fs\n", tempoCalculo);
  printf("Tempo de teste: %.3fs\n", tempoTeste);
  printf("Tempo total: %.3fs\n", tempoInicializacao + tempoCalculo + tempoTeste);
  return 0;
}
```

Figura 2: Código paralelizado (usando GPU)

```
// Soma os vetores de entrada e armazena o resultado no dispositivo.
#pragma omp target
#pragma omp loop
for (int indice = 0; indice < tamanhoVetor; indice++){
  vetorSoma[indice] = vetorA[indice] + vetorB[indice];
}
```

5. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho do experimento.

Figura 3: Resultado usando paralelização com CPU

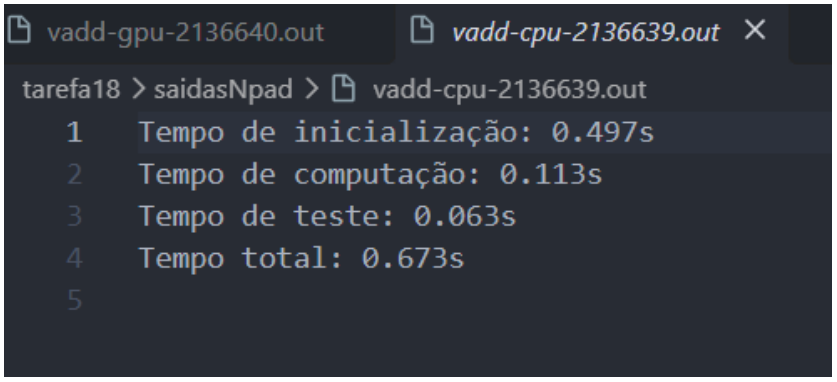
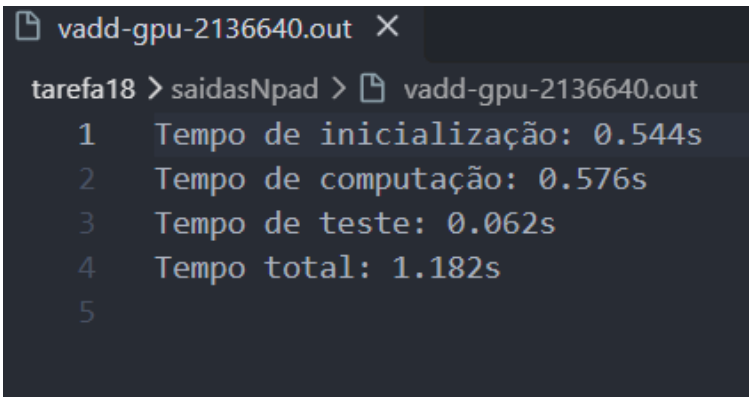


Figura 4: Resultado usando paralelização com GPU



Os resultados mostram que, nesta implementação, a versão em CPU foi mais rápida que a versão com GPU.

Isso não significa que a GPU seja intrinsicamente pior para soma de vetores. O ponto central é a transferência de dados. A GPU só executa o laço de soma, enquanto os vetores são inicializados e validados na memória principal (CPU). Ao entrar em *#pragma omp target*, o OpenMP precisa transferir os vetores necessários para a GPU e, depois, trazer *vetorSoma* de volta para que a CPU possa testá-lo.

Para 100 milhões de float, cada vetor ocupa aproximadamente 400MB. Só os vetores de entrada e resultado representam cerca de 1,2GB movimentados entre CPU e GPU. Essa cópia pela conexão CPU–GPU tende a custar mais do que a própria operação *A[i] + B[i]*, que é extremamente simples e rápida.

Assim, o tempo chamado de “computação” na versão GPU (0,576 s) não mede somente o cálculo paralelo, ele inclui o custo implícito de enviar dados ao dispositivo e recuperar o resultado. Na CPU, os dados já estão no mesmo espaço de memória do processamento, então esse custo não existe.

Como solução, propõe-se manter os dados no dispositivo durante várias etapas de processamento, transferindo-os apenas quando necessário, por meio de regiões target data. Também é recomendado executar a inicialização e a validação na GPU e aplicar múltiplas operações aos mesmos vetores antes de copiá-los de volta à CPU.